

Clase 10

Transformar datos en mejores variables

Diplomado de Python y Análisis de Datos

Universidad Marista

2026

Objetivos de la sesión

Al finalizar la clase, el estudiante será capaz de:

- comprender qué es Feature Engineering;
- identificar features útiles, inútiles y peligrosas;
- transformar variables numéricas;
- escalar variables;
- codificar variables categóricas;
- utilizar One-Hot Encoding;
- crear variables derivadas;
- construir ratios e interacciones;
- crear variables temporales;
- aplicar transformaciones logarítmicas;
- evitar Data Leakage durante preprocessing;
- utilizar ColumnTransformer;
- construir un Pipeline de Scikit-learn;
- comparar modelos antes y después del Feature Engineering.

Tiempo	Bloque
0:00–0:15	¿Qué es Feature Engineering?
0:15–0:35	Selección de variables y variables peligrosas
0:35–1:00	Escalamiento: StandardScaler y MinMaxScaler
1:00–1:25	Variables categóricas y One-Hot Encoding
1:25–1:45	Binning y transformaciones numéricas
1:45–1:55	Descanso
1:55–2:20	Interacciones, ratios y Polynomial Features
2:20–2:40	Variables temporales y de negocio
2:40–3:05	Data Leakage en preprocessing
3:05–3:25	ColumnTransformer + Pipeline
3:25–3:45	Live Coding: antes vs después
3:45–3:55	Feature Engineering Challenge
3:55–4:00	Conclusiones y puente a Regresión

Hasta ahora...

Ya sabemos:

$$X \rightarrow \text{Modelo} \rightarrow \hat{y}$$

Pero hay una pregunta que todavía no hemos estudiado profundamente.

Pregunta

¿De dónde salen las variables que colocamos dentro de X ?

Un algoritmo solamente ve números

Supongamos:

Fecha	Zona	Precio
2026-08-01	Centro	2.8M
2026-08-02	Norte	2.1M

Para nosotros:

“Centro”

tiene significado.

Para muchos algoritmos:

necesitamos una representación numérica.

Feature Engineering

Feature Engineering es el proceso de:

crear, seleccionar o transformar variables

para representar mejor:

la información relevante del problema.

Conceptualmente:

Datos crudos → *Features* → *Modelo*

Una buena representación importa

Supongamos que tenemos:

metros

y:

habitaciones

Podemos crear:

$$\text{metros_por_habitacion} = \frac{\text{metros}}{\text{habitaciones}}$$

Esta nueva variable podría capturar:

espacio disponible por habitación.

Feature Engineering no es inventar datos

No estamos creando información de la nada.

Estamos:

- reorganizando;
- transformando;
- representando;
- combinando;
- simplificando

información que ya existe.

¿Toda columna debe entrar al modelo?

No.

Supongamos:

id	nombre	metros	precio
1001	Ana	120	2.5M
1002	Pedro	90	1.9M

¿Debe entrar:

id

como feature?

No necesariamente.

Variables identificadoras

Ejemplos:

- ID de cliente;
- número de expediente;
- UUID;
- número de factura;
- nombre completo.

Normalmente sirven para:

identificar

no necesariamente para:

predecir.

Antes de utilizar una columna:

¿Existe una razón por la cual esta variable debería contener información predictiva?

No todas las columnas son features.

Variable peligrosa

Queremos predecir:

Churn

y utilizamos:

fecha_cancelacion

como feature.

Problema

La variable prácticamente contiene la respuesta.

Esto es:

Data Leakage

Para cada feature preguntar:

¿Esta información existiría realmente en el momento de hacer la predicción?

Si:

No

debemos investigar posible leakage.

Escalas diferentes

Supongamos:

$$Edad = 35$$

pero:

$$Ingreso = 800\,000$$

y:

$$Visitas = 5$$

Las escalas son muy diferentes.

¿Por qué importa la escala?

Algunos algoritmos utilizan:

- distancia;
- gradientes;
- regularización.

Ejemplos sensibles:

- KNN;
- K-Means;
- SVM;
- redes neuronales;
- modelos regularizados.

Ejemplo de distancia

Tenemos dos diferencias:

$$\Delta Edad = 10$$

$$\Delta Ingreso = 300\,000$$

En distancia euclidiana:

$$d = \sqrt{10^2 + 300000^2}$$

¿Qué variable dominará?

Ingreso

StandardScaler transforma aproximadamente:

$$z = \frac{x - \mu}{\sigma}$$

Después de transformar:

media cercana a 0

y:

desviación estándar cercana a 1.

Ejemplo

Tenemos:

$$x = 120$$

$$\mu = 100$$

$$\sigma = 10$$

Entonces:

$$z = \frac{120 - 100}{10}$$

$$\boxed{z = 2}$$

StandardScaler

```
1 from sklearn.preprocessing import (  
2     StandardScaler  
3 )  
4  
5 scaler = StandardScaler()  
6  
7 X_train_scaled = scaler.fit_transform(  
8     X_train  
9 )  
10  
11 X_test_scaled = scaler.transform(  
12     X_test  
13 )
```

fit_transform vs transform

En Train:

`fit_transform()`

porque:

aprendemos media y desviación

y transformamos.

En Test:

`transform()`

porque utilizamos exactamente los parámetros aprendidos en Train.

```
scaler.fit_transform(X_test)
```

¿Por qué?

Porque estaríamos permitiendo que Test influya en:

μ

y:

σ

utilizados en su propia transformación.

Otra transformación común:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Normalmente lleva los datos a:

$$[0, 1]$$

Ejemplo Min-Max

Supongamos:

$$x_{\text{mín}} = 0$$

$$x_{\text{máx}} = 100$$

y:

$$x = 40$$

Entonces:

$$x' = \frac{40}{100}$$

$$x' = 0.4$$

MinMaxScaler

```
1 from sklearn.preprocessing import (  
2     MinMaxScaler  
3 )  
4  
5 scaler = MinMaxScaler()  
6  
7 X_train_scaled = scaler.fit_transform(  
8     X_train  
9 )  
10  
11 X_test_scaled = scaler.transform(  
12     X_test  
13 )
```


StandardScaler vs MinMaxScaler

	StandardScaler	MinMaxScaler
Centro	0	No necesariamente
Escala	Desviación 1	[0, 1]
Outliers	Puede afectarse	Muy sensible

No existe un escalador universalmente mejor.

¿Todos los modelos necesitan escalamiento?

No.

Modelos basados en árboles como:

- Decision Tree;
- Random Forest;
- Gradient Boosting

generalmente son mucho menos sensibles a la escala.

Idea

El preprocessing depende del algoritmo.

El problema de las categorías

Tenemos:

Zona

con:

Centro
Norte
Sur

Muchos modelos requieren una representación numérica.

Una mala idea

Podríamos escribir:

$$\textit{Centro} = 1$$

$$\textit{Norte} = 2$$

$$\textit{Sur} = 3$$

Pero esto podría sugerir:

$$\textit{Sur} > \textit{Norte} > \textit{Centro}$$

aunque no exista ese orden.

One-Hot Encoding

Transformamos:

Zona

en variables binarias:

Centro	Norte	Sur
1	0	0
0	1	0
0	0	1

Con Pandas

```
1 df_encoded = pd.get_dummies(  
2     df,  
3     columns=[  
4         "zona"  
5     ],  
6     drop_first=True  
7 )  
8  
9 print(  
10     df_encoded.head()  
11 )
```

Con Scikit-learn

```
1 from sklearn.preprocessing import (  
2     OneHotEncoder  
3 )  
4  
5 encoder = OneHotEncoder(  
6     handle_unknown="ignore",  
7     sparse_output=False  
8 )  
9  
10 encoder.fit(  
11     X_train[["zona"]]  
12 )  
13  
14 zona_train = encoder.transform(  
15     X_train[["zona"]]  
16 )  
17  
18 zona_test = encoder.transform(  
19     X_test[["zona"]]  
20 )
```

Supongamos que Train tiene:

Centro, Norte, Sur

pero Test contiene:

Poniente

Con:

handle_unknown = "ignore"

evitamos un error directo por esa categoría desconocida.

Categorías con orden

Algunas categorías sí tienen orden.

Ejemplo:

Bajo
Medio
Alto

Aquí tiene sentido una representación como:

$$Bajo = 0$$

$$Medio = 1$$

$$Alto = 2$$

Nominal vs ordinal

Nominal

Sin orden natural.

- ciudad;
- color;
- producto.

Ordinal

Con orden natural.

- bajo/medio/alto;
- satisfacción;
- nivel educativo.

OrdinalEncoder

```
1 from sklearn.preprocessing import (  
2     OrdinalEncoder  
3 )  
4  
5 encoder = OrdinalEncoder(  
6     categories=[  
7         [  
8             "Bajo",  
9             "Medio",  
10            "Alto"  
11        ]  
12    ]  
13 )
```

¿Y LabelEncoder?

LabelEncoder suele utilizarse principalmente para:

y

es decir:

las etiquetas objetivo.

Para columnas categóricas dentro de X normalmente preferimos:

- OneHotEncoder;
- OrdinalEncoder.

Binning consiste en convertir una variable continua en intervalos.

Ejemplo:

Edad

se convierte en:

Joven
Adulto
Mayor

Binning manual

```
1 df["grupo_edad"] = pd.cut(  
2     df["edad"],  
3     bins=[  
4         0,  
5         25,  
6         40,  
7         60,  
8         100  
9     ],  
10    labels=[  
11        "18-25",  
12        "26-40",  
13        "41-60",  
14        "60+"  
15    ]  
16 )
```

¿Por qué utilizar binning?

Puede ayudar a:

- simplificar relaciones;
- crear segmentos interpretables;
- capturar umbrales;
- incorporar conocimiento del negocio.

Pero:

Cuidado

También perdemos información al convertir números continuos en categorías.

Variables muy sesgadas

Supongamos ingresos:

10000, 15000, 20000, 25000, 1000000

Existe una cola muy larga.

Podemos estudiar:

$$\log(1 + x)$$

Transformación log

```
1 import numpy as np
2
3 df["ingreso_log"] = np.log1p(
4     df["ingreso"]
5 )
```

$\log_{1p}(x)$ calcula:

$$\log(1 + x)$$

y permite manejar:

$$x = 0$$

sin obtener $\log(0)$.

¿Qué logra el log?

Puede:

- reducir asimetría;
- comprimir valores extremos;
- facilitar algunas relaciones;
- estabilizar escalas.

Cuidado

No debemos transformar variables automáticamente sin una razón.

DESCANSO

10 minutos

Al regresar:

¿Cómo crear información nueva a partir de las variables existentes?

Una variable puede depender de otra

Supongamos:

metros

y:

zona

Quizá:

10 m²

adicionales no valen lo mismo:

en Centro

que:

en una zona periférica.

Una interacción estudia cómo el efecto de una variable puede depender de otra.

Ejemplo:

$$x_1 x_2$$

Podemos crear:

$$\textit{metros} \times \textit{habitaciones}$$

aunque la interpretación depende del problema.

Muchas features útiles provienen de razones.

Ejemplos:

$$\text{metros_por_habitacion} = \frac{\text{metros}}{\text{habitaciones}}$$

$$\text{gasto_promedio} = \frac{\text{gasto_total}}{\text{numero_compras}}$$

Crear ratios

```
1 df["metros_por_habitacion"] = (  
2     df["metros"]  
3     /  
4     df["habitaciones"]  
5 )  
6  
7 df["gasto_promedio"] = (  
8     df["gasto_total"]  
9     /  
10    df["numero_compras"]  
11 )
```

Cuidado con división entre cero

Si:

$$\text{numero_compras} = 0$$

entonces:

$$\frac{\text{gasto}}{\text{numero_compras}}$$

no está definido.

Siempre debemos validar las reglas de negocio.

Relaciones no lineales

Supongamos que:

 y

no cambia linealmente con:

 x

Podríamos incluir:

 x^2

o:

 $x_1 x_2$

como features adicionales.

Ejemplo polinomial

Un modelo:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

puede representar curvatura.

Seguimos utilizando un modelo lineal en los parámetros, pero con variables transformadas.

PolynomialFeatures

```
1 from sklearn.preprocessing import (  
2     PolynomialFeatures  
3 )  
4  
5 poly = PolynomialFeatures(  
6     degree=2,  
7     include_bias=False  
8 )  
9  
10 X_poly = poly.fit_transform(  
11     X[  
12         [  
13             "metros",  
14             "antiguedad"  
15         ]  
16     ]  
17 )
```

¿Qué crea degree=2?

Para:

x_1, x_2

puede crear:

x_1

x_2

x_1^2

$x_1 x_2$

x_2^2

Más features también pueden ser peligrosas

Si agregamos demasiadas variables:

$p \uparrow$

podemos aumentar:

- complejidad;
- ruido;
- costo;
- riesgo de overfitting.

Regla

Más features no significa automáticamente mejor modelo.

Una fecha contiene muchas features

Tenemos:

2026 — 08 — 21

Podemos obtener:

- año;
- mes;
- día;
- día de semana;
- trimestre;
- fin de semana.

Features temporales

```
1 df["fecha"] = pd.to_datetime(  
2     df["fecha"]  
3 )  
4  
5 df["anio"] = (  
6     df["fecha"].dt.year  
7 )  
8  
9 df["mes"] = (  
10     df["fecha"].dt.month  
11 )  
12  
13 df["dia_semana"] = (  
14     df["fecha"].dt.dayofweek  
15 )  
16  
17 df["trimestre"] = (  
18     df["fecha"].dt.quarter  
19 )
```

Fin de semana

```
1 df["fin_semana"] = (  
2     df["fecha"]  
3     .dt.dayofweek  
4     .isin([  
5         5,  
6         6  
7     ])  
8     .astype(int)  
9 )
```

Ahora:

1

significa fin de semana.

Feature Engineering también utiliza conocimiento del dominio.

Ejemplos:

- antigüedad del cliente;
- días desde última compra;
- gasto promedio;
- tasa de conversión;
- frecuencia de visitas;
- ocupación por habitación.

RFM como ejemplo

En clientes podemos construir:

$R = \text{Recency}$

$F = \text{Frequency}$

$M = \text{Monetary}$

Estas tres variables resumen:

- qué tan reciente compró;
- qué tan seguido compra;
- cuánto gasta.

Leakage también ocurre al transformar

Supongamos que calculamos:

$$\mu$$

y:

$$\sigma$$

utilizando:

$$Train + Test$$

antes de escalar.

Entonces Test influyó en el preprocessing.

Leakage

Orden correcto

Primero:

Train/ Test

Después:

fit preprocessing en Train

Después:

transform Train

y:

transform Test

utilizando solamente lo aprendido en Train.

Datos



Scaler.fit(Datos)



Train/ Test

Problema

El scaler ya vio información que terminará en Test.

Datos



Train/ Test



Scaler.fit(Train)



Transform(Train) Transform(Test)

Datasets mixtos

Nuestro dataset puede contener:

Variable	Tipo
metros	numérica
edad	numérica
zona	categorica
tipo	categorica

Necesitamos diferentes transformaciones.

La solución

Podemos decir:

Variables numéricas

→ *StandardScaler*

y:

Variables categóricas

→ *OneHotEncoder*

utilizando:

ColumnTransformer

Definir columnas

```
1 numericas = [  
2     "metros",  
3     "habitaciones",  
4     "banios",  
5     "antiguedad"  
6 ]  
7  
8 categoricas = [  
9     "zona",  
10    "tipo_vivienda"  
11 ]
```

ColumnTransformer

```
1 from sklearn.compose import (  
2     ColumnTransformer  
3 )  
4  
5 from sklearn.preprocessing import (  
6     StandardScaler,  
7     OneHotEncoder  
8 )  
9  
10 preprocesador = ColumnTransformer(  
11     transformers=[  
12         (  
13             "num",  
14             StandardScaler(),  
15             numericas  
16         ),  
17         (  
18             "cat",  
19             OneHotEncoder(  
20                 handle_unknown="ignore"  
21             ),  
22             categoricas
```

Pipeline

Un Pipeline permite conectar:

Preprocessing

con:

Modelo

para evitar procesos manuales inconsistentes.

Conceptualmente:

$X \rightarrow \text{Transformar} \rightarrow \text{Modelo} \rightarrow \text{Predicción}$

Pipeline completo

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.linear_model import (
3     LinearRegression
4 )
5
6 pipeline = Pipeline(
7     steps=[
8         (
9             "preprocesamiento",
10            preprocesador
11        ),
12        (
13            "modelo",
14            LinearRegression()
15        )
16    ]
17 )
```

Entrenar Pipeline

```
1 pipeline.fit(  
2     X_train,  
3     y_train  
4 )  
5  
6 predicciones = pipeline.predict(  
7     X_test  
8 )
```

El Pipeline se encarga de:

- 1 aprender preprocessing en Train;
- 2 transformar;
- 3 entrenar el modelo;
- 4 transformar correctamente Test;
- 5 predecir.

Con Pipeline reducimos el riesgo de:

- olvidar transformaciones;
- transformar Train y Test diferente;
- introducir leakage accidental;
- romper el flujo al llevar el modelo a producción.

Caso práctico

Utilizaremos:

`viviendas_features.csv`

Variables:

- metros;
- habitaciones;
- baños;
- antigüedad;
- zona;
- tipo_vivienda;
- fecha_publicacion;
- precio.

Objetivo

Compararemos dos experimentos.

Modelo A

con variables básicas.

Contra:

Modelo B

con Feature Engineering.

Paso 1: cargar

```
1 import pandas as pd
2
3 df = pd.read_csv(
4     "viviendas_features.csv"
5 )
6
7 print(
8     df.head()
9 )
10
11 df.info()
```

Paso 2: limpiar fecha

```
1 df["fecha_publicacion"] = (  
2     pd.to_datetime(  
3         df["fecha_publicacion"]  
4     )  
5 )
```

Paso 3: variables temporales

```
1 df["mes_publicacion"] = (  
2     df["fecha_publicacion"]  
3     .dt.month  
4 )  
5  
6 df["dia_semana"] = (  
7     df["fecha_publicacion"]  
8     .dt.dayofweek  
9 )
```

Paso 4: nuevas features

```
1 df["metros_por_habitacion"] = (  
2     df["metros"]  
3     /  
4     df["habitaciones"]  
5 )  
6  
7 df["banios_por_habitacion"] = (  
8     df["banios"]  
9     /  
10    df["habitaciones"]  
11 )
```

Validar antes de dividir

Antes de crear ratios revisar:

habitaciones = 0?

También:

banios < 0?

metros ≤ 0?

Feature Engineering no reemplaza la validación de datos.

Paso 5: X e y

```
1 X = df.drop(  
2     columns=[  
3         "precio",  
4         "fecha_publicacion"  
5     ]  
6 )  
7  
8 y = df[  
9     "precio"  
10 ]
```

Paso 6: Train/Test

```
1 from sklearn.model_selection import (  
2     train_test_split  
3 )  
4  
5 X_train, X_test, y_train, y_test = (  
6     train_test_split(  
7         X,  
8         y,  
9         test_size=0.20,  
10        random_state=42  
11    )  
12 )
```

Paso 7: columnas

```
1 numericas = [  
2     "metros",  
3     "habitaciones",  
4     "banios",  
5     "antiguedad",  
6     "mes_publicacion",  
7     "dia_semana",  
8     "metros_por_habitacion",  
9     "banios_por_habitacion"  
10 ]  
11  
12 categoricas = [  
13     "zona",  
14     "tipo_vivienda"  
15 ]
```


Paso 8: preprocessing

```
1 preprocesador = ColumnTransformer(  
2     transformers=[  
3         (  
4             "numericas",  
5             StandardScaler(),  
6             numericas  
7         ),  
8         (  
9             "categoricas",  
10            OneHotEncoder(  
11                handle_unknown="ignore"  
12            ),  
13            categoricas  
14        )  
15     ]  
16 )
```

Paso 9: pipeline

```
1 pipeline = Pipeline(  
2     steps=[  
3         (  
4             "prep",  
5             preprocesador  
6         ),  
7         (  
8             "modelo",  
9             LinearRegression()  
10        )  
11    ]  
12 )  
13  
14 pipeline.fit(  
15     X_train,  
16     y_train  
17 )
```

Paso 10: predecir

```
1 pred = pipeline.predict(  
2     X_test  
3 )
```

Ahora:

$\hat{y}$

fue producido después de aplicar automáticamente todas las transformaciones correspondientes.

Paso 11: evaluar

```
1 from sklearn.metrics import (  
2     mean_absolute_error,  
3     mean_squared_error,  
4     r2_score  
5 )  
6  
7 mae = mean_absolute_error(  
8     y_test,  
9     pred  
10 )  
11  
12 rmse = (  
13     mean_squared_error(  
14         y_test,  
15         pred  
16     ) ** 0.5  
17 )  
18  
19 r2 = r2_score(  
20     y_test,  
21     pred  
22 )
```

Modelo inicial:

$$X = \{\textit{metros}, \textit{habitaciones}, \textit{banios}, \textit{antiguedad}\}$$

Modelo mejorado:

$$X = \{\textit{variables originales} + \textit{categorias} + \textit{ratios} + \textit{fecha}\}$$

Ejemplo:

Modelo	MAE	R^2
Original	220000	0.72
Feature Engineering	145000	0.86

Pregunta

¿Qué variables nuevas pudieron aportar información?

Pero también puede empeorar

Supongamos:

$$MAE_{antes} = 150000$$

$$MAE_{despues} = 175000$$

Importante

Una feature nueva debe demostrar que aporta valor.

No debemos conservar variables solamente porque parecen interesantes.

¿Qué variables utiliza más un modelo?

Algunos modelos ofrecen medidas de:

Feature Importance

Por ejemplo:

- árboles;
- Random Forest;
- Gradient Boosting.

Ejemplo rápido

```
1 from sklearn.ensemble import (  
2     RandomForestRegressor  
3 )  
4  
5 modelo_rf = RandomForestRegressor(  
6     n_estimators=100,  
7     random_state=42  
8 )  
9  
10 modelo_rf.fit(  
11     X_train_num,  
12     y_train  
13 )  
14  
15 print(  
16     modelo_rf.feature_importances_  
17 )
```

Cuidado con importancia

Importancia no significa:

causalidad

Además, diferentes métodos de importancia tienen diferentes propiedades.

Más adelante veremos:

SHAP

para explicabilidad.

Feature Engineering Challenge

¿Podemos mejorar un modelo sin cambiar el algoritmo?

Regla del reto

Todos utilizarán:

LinearRegression

No pueden cambiar el algoritmo.
Solamente pueden modificar:

X

El objetivo:

$MAE \downarrow$

`clientes_features.csv`

Variables:

- edad;
- ingreso;
- visitas;
- compras;
- gasto_total;
- ciudad;
- fecha_registro;
- gasto_siguiente_mes.

Target

Queremos predecir:

gasto_siguiente_mes

Por lo tanto:

$y = \text{gasto_siguiente_mes}$

Fase 1

Construir modelo base usando solamente:

- edad;
- ingreso;
- visitas;
- compras;
- gasto_total.

Registrar:

$$MAE_{base}$$

Crear al menos:

- 1 una variable ratio;
- 2 una variable temporal;
- 3 una transformación;
- 4 una variable categórica codificada.

$$\text{gasto_por_compra} = \frac{\text{gasto_total}}{\text{compras}}$$

$$\text{visitas_por_compra} = \frac{\text{visitas}}{\text{compras}}$$

antigüedad_cliente

ingreso_log

ciudad $\rightarrow$ *OneHot*

Crear:

Pipeline

con:

- StandardScaler para numéricas;
- OneHotEncoder para categóricas;
- LinearRegression.

Fase 4

Comparar:

$$MAE_{base}$$

contra:

$$MAE_{features}$$

Calcular:

$$Mejora = MAE_{base} - MAE_{features}$$

Cada equipo deberá responder:

- ① ¿Qué feature nueva ayudó más?
- ② ¿Qué feature no ayudó?
- ③ ¿Alguna produjo overfitting?
- ④ ¿Existe riesgo de leakage?
- ⑤ ¿Qué variables conservarían?

“Voy a escalar todo el dataset antes del Train/Test.”

Incorrecto

Puede introducir Data Leakage.

“Centro=1, Norte=2, Sur=3.”

Problema

Estamos introduciendo un orden artificial si la variable es nominal.

“Cuantas más features tenga, mejor será el modelo.”

Incorrecto

Las variables irrelevantes pueden aumentar ruido y complejidad.

“Si una variable mejora mucho el modelo, debe ser excelente.”

Primero preguntar:

¿Existe en el momento real de predicción?

Puede ser:

Leakage

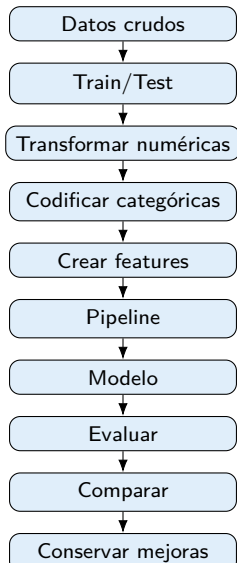
“Todos los algoritmos necesitan StandardScaler.”

No.

El preprocessing depende de:

- algoritmo;
- distribución;
- objetivo;
- tipo de datos.

Pipeline de Feature Engineering



Herramientas que vimos

Objetivo	Herramienta
Estandarizar	<code>StandardScaler()</code>
Normalizar	<code>MinMaxScaler()</code>
One-Hot	<code>OneHotEncoder()</code>
Ordinal	<code>OrdinalEncoder()</code>
Binning	<code>pd.cut()</code>
Log	<code>np.log1p()</code>
Polinomios	<code>PolynomialFeatures()</code>
Transformar columnas	<code>ColumnTransformer()</code>
Pipeline	<code>Pipeline()</code>

15 ideas que deben recordar

- 1 Los modelos aprenden de la representación que les damos.
- 2 No toda columna debe ser una feature.
- 3 IDs normalmente no aportan significado predictivo directo.
- 4 Siempre debemos investigar leakage.
- 5 La escala importa para algunos algoritmos.
- 6 StandardScaler centra y escala.
- 7 MinMaxScaler lleva normalmente a $[0, 1]$.
- 8 One-Hot Encoding sirve para categorías nominales.
- 9 Ordinal Encoding requiere un orden real.
- 10 Binning simplifica pero pierde información.
- 11 Transformaciones logarítmicas pueden reducir asimetría.
- 12 Ratios pueden capturar información de negocio.
- 13 Interacciones permiten representar relaciones más complejas.
- 14 El preprocessing debe aprenderse solamente con Train.
- 15 Pipeline ayuda a construir procesos reproducibles y seguros.

Pregunta de salida 1

Tenemos:

$$Edad = 30$$

$$Ingreso = 800000$$

y utilizaremos:

KNN

Pregunta

¿Qué transformación investigaríamos primero?

Una posibilidad:

Escalamiento

Pregunta de salida 2

Tenemos:

Centro
Norte
Sur

sin orden natural.

¿Qué utilizaríamos?

One – Hot Encoding

Pregunta de salida 3

Calculamos:

StandardScaler.fit(X)

antes de separar Train/Test.

¿Qué problema existe?

Data Leakage

Ya tenemos mejores variables

Ahora podemos construir:

X

de mejor calidad.

Pero todavía falta comprender profundamente:

¿cómo aprende una regresión?

Clase 11

Regresión

Predicción de variables numéricas

Veremos:

- regresión lineal simple;
- mínimos cuadrados;
- pendiente e intercepto;
- regresión múltiple;
- interpretación de coeficientes;
- supuestos;
- residuos;
- MAE;

Feature Engineering

El algoritmo importa.

Pero también importa

cómo representamos el problema.

Diplomado de Python y Análisis de Datos

Universidad Marista